

matlab_simulation (MATLAB)

=====

FILE: src/matlab/startup.m

```
-----
function startup()
%STARTUP Project startup for ERCOT Electrolyzer Sim Engine
%
% This MATLAB project is a pure simulation engine.
% - No data is owned here
% - No plots are created here
% - No configs are defined here
% - Only numeric simulation code lives on the path

    fprintf('[startup] Initializing ERCOT Electrolyzer Sim Engine...\n');

%-----
% Resolve paths
%-----
% This file: "<repo>/src/matlab/startup.m"
thisFile      = mfilename('fullpath');
matlabRoot    = fileparts(thisFile);           % <repo>/src/matlab
repoRoot      = fileparts(fileparts(matlabRoot)); % <repo>

%-----
% Export repo root for IO and batch jobs
%-----
setenv('ERCOT_SIM_ROOT', repoRoot);
fprintf('[startup] ERCOT_SIM_ROOT set to:\n  %s\n', repoRoot);

%-----
% Safety check: ensure expected MATLAB structure exists
%-----
requiredDirs = {
    'engine'
    'models'
    'sim'
    'utils'
    'io'
};

for i = 1:numel(requiredDirs)
    dirPath = fullfile(matlabRoot, requiredDirs{i});
    if ~isfolder(dirPath)
        error('[startup] Required directory missing: %s', dirPath);
    end
end

%-----
% Reset MATLAB path (project hygiene)
%-----
restoredefaultpath;

%-----
% Add ONLY execution-relevant directories
```

```
%-----
addpath( ...
    matlabRoot, ...
    fullfile(matlabRoot, 'engine'), ...
    fullfile(matlabRoot, 'models'), ...
    fullfile(matlabRoot, 'sim'), ...
    fullfile(matlabRoot, 'utils'), ...
    fullfile(matlabRoot, 'io') ...
);
```

```
%-----
% Explicitly DO NOT add (documentation-by-code)
%-----
% tools/          -> developer-only helpers
% configs/        -> declarative inputs (CSV, TOML, etc.)
% data/           -> immutable inputs (accessed only by io/)
% scripts/        -> orchestration only
% outputs/        -> artifacts only
% docs/           -> no executable code
```

```
%-----
% Optional: numeric & display defaults (safe)
%-----
format long g
warning('off', 'MATLAB:dispatcher:nameConflict');
```

```
fprintf('[startup] MATLAB path configured successfully.\n');
```

```
end
```

```
# FILE: src/matlab/sim/run_sim.m
```

```
%=====
%  INITIALIZE
% =====
clc; clear;
```

```
%=====
%  LOAD INPUTS AND CASES
% =====
[inputs, cases] = load_inputs_and_cases();
```

```
signals = compute_clean_energy_ratio(inputs);
emissions_kernel = compute_grid_emissions_kernel(inputs, signals);
electrolyzer_system = define_electrolyzer_system();
stack = electrolyzer_system.stack;
```

```
%=====
%  SIMULATION WINDOW
% =====
run_mode = "all";
years_to_simulate = 2025;
```

```
switch run_mode
```

```

        case "all"
            sim_idx = true(size(inputs.time));
        case "year"
            sim_idx = ismember(year(inputs.time), years_to_simulate);
    end

    sim_steps = find(sim_idx);
    n_steps_sim = numel(sim_steps);
    dt_hr = hours(inputs.time(2) - inputs.time(1));

    fprintf("Simulating %d timesteps (%s mode)\n", n_steps_sim, run_mode);

    %% =====
    %   HDF5 STREAMING SETUP
    %   =====
    n_cases = height(cases);
    chunkSize = 20;

    project_root = get_project_root();
    outDir = fullfile(project_root, 'outputs', 'results');
    outFile = fullfile(outDir, 'results_timeseries.h5');

    if ~isfolder(outDir)
        mkdir(outDir);
    end

    init_results_hdf5_chunked( ...
        outFile, ...
        n_cases, ...
        inputs.time(sim_steps), ...
        chunkSize);

    %% =====
    %   MAIN SERIAL LOOP (STREAMING)
    %   =====
    buffer = struct([]);
    chunkStart = 1;
    startTime = tic;
    lastPrint = tic;

    for ii = 1:n_cases

        % --- Progress update (single line)
        if toc(lastPrint) > 0.1 || ii == n_cases
            fprintf('\rElapsed: %s | Cases remaining: %d', ...
                format_time(toc(startTime)), n_cases - ii + 1);
            drawnow limitrate
            lastPrint = tic;
        end

        % --- Case setup
        current_case = table2struct(cases(ii,:), 'ToScalar', true);
        state = initialize_electrolyzer_system_state();
    end

```

```

P_grid_kW = zeros(n_steps_sim,1);
h2_kgph   = zeros(n_steps_sim,1);
cost      = zeros(n_steps_sim,1);

% --- Timestep loop
for jj = 1:n_steps_sim
    t = sim_steps(jj);

    state = update_electrolyzer_system_state( ...
        state, current_case, stack, signals, t, false);

    phys = electrolyzer_system.physics(state.P_stack_cmd_kW);

    P_grid_kW(jj) = phys.system.P_grid_kW;
    h2_kgph(jj)   = phys.psa.mH2_net_kgph;
    cost(jj)      = phys.system.P_grid_kW * dt_hr * signals.price(t)/1000;
end

% --- Assemble sim struct
local_sim = struct();
local_sim.time      = inputs.time(sim_steps);
local_sim.P_grid_kW = P_grid_kW;
local_sim.h2_kgph   = h2_kgph;
local_sim.cost      = cost;

% --- Emissions
emissions_i = electrolyzer_emissions_from_kernel( ...
    inputs, emissions_kernel, local_sim, sim_steps);

% --- Totals
local_sim.total_energy_MWh = sum(P_grid_kW) * dt_hr / 1000;
local_sim.total_cost      = sum(cost);
local_sim.total_h2_kg     = sum(h2_kgph) * dt_hr;

% --- Collect buffer
buffer(end+1).ID      = current_case.ID;
buffer(end).sim       = local_sim;
buffer(end).emissions = emissions_i;
buffer(end).state     = state;

% --- Flush chunk
if numel(buffer) == chunkSize || ii == n_cases
    write_results_hdf5_chunk(outFile, buffer, chunkStart);
    chunkStart = chunkStart + numel(buffer);
    buffer = struct([]);
end
end

fprintf('\nCompleted all cases. Elapsed: %s\n', ...
    format_time(toc(startTime)));

%% =====
%   UTIL
%   =====

```

```

function s = format_time(sec)
    h = floor(sec/3600);
    sec = sec - 3600*h;
    m = floor(sec/60);
    s2 = floor(sec - 60*m);
    s = sprintf('%02dh:%02dm:%02ds', h, m, s2);
end

# FILE: src/matlab/sim/post_sim.m
-----
%% Calculate Moving Averages
% Inputs
P_grid_kW = results.sim.P_grid_kW;
h2_kgph = results.sim.h2_kgph;
electricity_price = signals.price;
clean_ratio = signals.clean_ratio;
co2_kg = results.emissions.co2_kg_per_timestep;
co2_kg_per_h2_kg = results.emissions.co2_per_kg_h2_timestep;
cost = results.sim.cost;
e_MWh = P_grid_kW * 0.25 / 1000;
h2_kg = h2_kgph * 0.25;

%% Calculate Moving Averages (D-day window)

D = 14; % <-- change this to any number of days you want

samples_per_day = 24 * 4; % 15-min timesteps → 96 per day
window = D * samples_per_day; % total samples in window

% Trailing moving averages (causal)
P_grid_kW_avg = movmean(P_grid_kW, [window-1 0]);
h2_kgph_avg = movmean(h2_kgph, [window-1 0]);
electricity_price_avg = movmean(electricity_price, [window-1 0]);
clean_ratio_avg = movmean(clean_ratio, [window-1 0]);
co2_kg_avg = movmean(co2_kg, [window-1 0]);
co2_kg_per_h2_kg_avg = movmean(co2_kg_per_h2_kg, [window-1 0]);
cost_avg = movmean(cost, [window-1 0]);
e_MWh_avg = movmean(e_MWh, [window-1 0]);
h2_kg_avg = movmean(h2_kg, [window-1 0]);

# FILE: src/matlab/io/load_inputs.m
-----
function inputs = load_inputs(input_dir)
%LOAD_INPUTS Load all ERCOT input time series from data/inputs
%
% inputs fields:
% .time -> datetime column (Nx1)
% .load
% .wind
% .solar
% .gas_cc
% .coal
% .nuclear
% .hydro

```

```

% .biomass
% .bess
% .wsl
% .price
% .dt_minutes
% .n_steps

%-----
% Resolve input directory robustly (repo-root relative)
%-----
if nargin == 0 || isempty(input_dir)
    repoRoot = getenv('ERCOT_SIM_ROOT');
    if isempty(repoRoot)
        error(['ERCOT_SIM_ROOT not set. ', ...
            'Did you open the MATLAB project or run startup.m?']);
    end
    input_dir = fullfile(repoRoot, 'data', 'inputs');
end

if ~isfolder(input_dir)
    error('Input directory does not exist: %s', input_dir);
end

%-----
% Define expected files
%-----
series_files = struct( ...
    'load',      'load.txt', ...
    'wind',      'wind.txt', ...
    'solar',     'solar.txt', ...
    'gas_cc',    'gas_cc.txt', ...
    'gas_other', 'gas_other.txt', ...
    'coal',      'coal.txt', ...
    'nuclear',   'nuclear.txt', ...
    'hydro',     'hydro.txt', ...
    'biomass',   'biomass.txt', ...
    'bess',      'bess.txt', ...
    'wsl',       'wsl.txt', ...
    'price',     'price.txt' ...
);

fields = fieldnames(series_files);

%-----
% Load first file to establish master time vector
%-----
first_file = fullfile(input_dir, series_files.(fields{1}));

if ~isfile(first_file)
    error('Missing input file: %s', first_file);
end

T = readtable(first_file, 'PreserveVariableNames', true);

```

```
time = datetime(T{:,1}, 'InputFormat', 'yyyy-MM-dd HH:mm');
inputs.time = time;
```

```
%-----
% Load remaining series and enforce time alignment
%-----
```

```
for k = 1:numel(fields)
    name = fields{k};
    file = fullfile(input_dir, series_files.(name));

    if ~isfile(file)
        error('Missing input file: %s', file);
    end

    T = readtable(file, 'PreserveVariableNames', true);

    t = datetime(T{:,1}, 'InputFormat', 'yyyy-MM-dd HH:mm');
    v = T{:,2};

    if ~isequal(t, time)
        error('Timestamp mismatch in file: %s', file);
    end

    inputs.(name) = v(:);
end
```

```
%-----
% Metadata
%-----
```

```
if numel(time) < 2
    error('Not enough time points to compute timestep.');
```

```
end

dt = time(2) - time(1); % duration
inputs.dt_hr = hours(dt);
inputs.n_steps = numel(time);
```

```
end
```

```
# FILE: src/matlab/io/load_supervisory_control_cases.m
```

```
-----
function cases = load_supervisory_control_cases()
```

```
%LOAD_SUPERVISORY_CONTROL_CASES
```

```
% Loads supervisory_control_cases.csv from configs/paper_cases
```

```
% Independent of current working directory
```

```
% -----
% Start from this file's location
% -----
```

```
this_file = mfilename('fullpath');
this_dir = fileparts(this_file);
```

```
% -----
% Walk upward until project root is found
```

```

% -----
project_root = '';
search_dir = this_dir;

while ~isempty(search_dir)
    candidate = fullfile( ...
        search_dir, ...
        'configs', ...
        'paper_cases', ...
        'supervisory_control_cases.csv' );

    if isfile(candidate)
        project_root = search_dir;
        break
    end

    % Move up one directory
    parent_dir = fileparts(search_dir);

    if strcmp(parent_dir, search_dir)
        break % hit filesystem root
    end

    search_dir = parent_dir;
end

% -----
% Error if not found
% -----
if isempty(project_root)
    error('Could not locate project root containing configs/paper_cases.');
```

```

end

% -----
% Load CSV
% -----
csv_path = fullfile( ...
    project_root, ...
    'configs', ...
    'paper_cases', ...
    'supervisory_control_cases.csv' );
```

```

opts = detectImportOptions(csv_path);
opts.VariableNamingRule = 'preserve';

cases = readtable(csv_path, opts);
```

```
end
```

```
# FILE: src/matlab/io/load_inputs_and_cases.m
```

```
-----
[ERROR] File not found: src/matlab/io/load_inputs_and_cases.m
```

```
# FILE: src/matlab/engine/compute_clean_energy_ratio.m
```

```

-----
function signals = compute_clean_energy_ratio(inputs)

% =====
% Clean Energy Ratio Calculator
%
% PURPOSE:
% Computes the time-resolved fraction of clean electricity supplied
% to the grid. Clean energy is defined as:
%
%   Clean = Renewable + Nuclear
%
% Non-renewable sources include fossil generation and BESS discharge.
% BESS charging (negative MW) is explicitly excluded.
%
% This signal is intended for emissions attribution and average
% grid-intensity modeling.
%
% DEFINITIONS:
%   - Renewable: wind, solar, hydro, biomass, WSL
%   - Clean (non-renewable): nuclear
%   - Non-renewable: gas, coal, BESS discharge
%
% NOTE:
%   BESS is treated as non-renewable when discharging and ignored
%   when charging. Storage is not credited as clean energy.
%
% UNITS:
%   All power signals are assumed to be in MW.
% =====

% -----
% Define generator groupings
% -----
renewable_types = {
    'wind'
    'solar'
    'hydro'
    'biomass'
    'wsl'
};

clean_types = {
    'nuclear'
};

nonrenewable_types = {
    'gas_cc'
    'gas_other'
    'coal'
    'bess'
};

```

```

% -----
% Initialize accumulators
% -----
N = inputs.n_steps;

total_MW      = zeros(N,1);
renewable_MW  = zeros(N,1);
clean_MW      = zeros(N,1);
nonrenewable_MW = zeros(N,1);

% -----
% Accumulate renewable generation
% -----
for ii = 1:numel(renewable_types)
    type = renewable_types{ii};

    if isfield(inputs, type)
        P = inputs.(type)(:);
        P = max(P, 0);

        renewable_MW = renewable_MW + P;
        total_MW = total_MW + P;
    end
end

% -----
% Accumulate clean (non-renewable) generation
% -----
for ii = 1:numel(clean_types)
    type = clean_types{ii};

    if isfield(inputs, type)
        P = inputs.(type)(:);
        P = max(P, 0);

        clean_MW = clean_MW + P;
        total_MW = total_MW + P;
    end
end

% -----
% Accumulate non-renewable generation
% (BESS discharge only)
% -----
for ii = 1:numel(nonrenewable_types)
    type = nonrenewable_types{ii};

    if isfield(inputs, type)
        P = inputs.(type)(:);

```

```

        % Only count discharge for BESS
        if strcmp(type, 'bess')
            P = max(P, 0);
        end

        nonrenewable_MW = nonrenewable_MW + P;
        total_MW        = total_MW        + P;
    end
end

% -----
% Compute clean energy ratios
% -----
clean_ratio = safe_divide(renewable_MW + clean_MW, total_MW);
renewable_ratio = safe_divide(renewable_MW, total_MW);

% -----
% Package outputs
% -----
signals.clean_ratio      = clean_ratio;
signals.renewable_ratio  = renewable_ratio;
signals.price            = inputs.price;

signals.total_MW         = total_MW;
signals.renewable_MW     = renewable_MW;
signals.clean_MW         = renewable_MW + clean_MW;
signals.nonrenewable_MW  = nonrenewable_MW;

end

# FILE: src/matlab/engine/compute_electricity_cost.m
-----
function signals = compute_electricity_cost(signals, results)

electricity_price = signals.price;

end

# FILE: src/matlab/engine/compute_grid_emissions_kernel.m
-----
function emissions = compute_grid_emissions_kernel(inputs, signals)

% =====
% Grid CO2 Emissions Kernel (Average Attribution)
%
% PURPOSE:
% Precomputes a time-indexed grid emissions intensity that maps
% electricity consumption to CO2 emissions:

```

```

%
%   C02(t) = Energy_consumed(t) [MWh] × alpha(t) [kg C02/MWh]
%
% This kernel depends ONLY on grid operation (inputs), not on
% electrolyzer behavior.
%
% OUTPUT:
%   emissions.alpha_kg_per_MWh    [n_steps x 1]
%
% NOTES:s
% - Kernel is defined per unit ENERGY (MWh), NOT power
% - Electrolyzer power may be supplied later in kW or MW
% =====

dt_hr = inputs.dt_hr;
N      = inputs.n_steps;

% -----
% Emission factors (kg C02 / MWh)
% -----
EF.coal      = 1000;
EF.ng_cc     = 450;
EF.ng_simple = 500;
EF.ng_steam  = 600;

% -----
% Extract fossil generation (MW)
% -----
P_coal  = max(inputs.coal, 0);
P_ng_cc = max(inputs.gas_cc, 0);
P_ng_all = max(inputs.gas_other, 0);

% -----
% Ramp-based NG decomposition
% -----
% Uses slow (steam) vs fast (simple-cycle) operational behavior
% to infer plausible contributions when unit-level data are absent
steam_window_steps = round(5 / dt_hr);

P_ng_baseline = movmean(P_ng_all, steam_window_steps);
P_ng_steam    = min(P_ng_baseline, P_ng_all);
P_ng_simple   = max(P_ng_all - P_ng_steam, 0);

% -----
% Fossil mix weights
% -----
P_fossil = P_coal + P_ng_cc + P_ng_simple + P_ng_steam;

w.coal      = safe_divide(P_coal,      P_fossil);
w.cc        = safe_divide(P_ng_cc,     P_fossil);
w.ng_simple = safe_divide(P_ng_simple, P_fossil);
w.ng_steam  = safe_divide(P_ng_steam,  P_fossil);

% -----

```

```

% Effective fossil CO2 intensity
% -----
% Result: kg CO2 / MWh of fossil electricity
EF_effective = ...
    w.coal      * EF.coal + ...
    w.cc        * EF.ng_cc + ...
    w.ng_simple * EF.ng_simple + ...
    w.ng_steam  * EF.ng_steam;

% -----
% Clean energy ratio (power-based)
% -----
clean_ratio = signals.clean_ratio;

% -----
% Final grid emissions kernel
% -----
% alpha(t): kg CO2 per MWh of electricity consumed
emissions.alpha_kg_per_MWh = (1 - clean_ratio) .* EF_effective;

% -----
% Diagnostics & traceability
% -----
emissions.EF_effective = EF_effective;

emissions.decomposition.P_ng_simple = P_ng_simple;
emissions.decomposition.P_ng_steam = P_ng_steam;

emissions.assumptions.EF = EF;
emissions.assumptions.method = ...
    ['Average attribution with ramp-based NG decomposition ' ...
    '(steam = slow energy, peaker = fast residual).'];

end

# FILE: src/matlab/engine/electrolyzer_emissions_from_kernel.m
-----
function emissions = electrolyzer_emissions_from_kernel( ...
    inputs, emissions_kernel, sim_results, sim_steps)
% =====
% Electrolyzer CO2 Emissions Attribution (Time-Window Aware)
%
% PURPOSE:
% Computes CO2 emissions attributable to electrolyzer electricity
% consumption using a precomputed grid emissions kernel, correctly
% aligned to the simulated timestep subset.
%
% REQUIRED INPUTS:
%   inputs.dt_hr
%   sim_results.P_grid_kW [n_sim_steps x 1]
%   emissions_kernel.alpha_kg_per_MWh [n_total_steps x 1]
%   sim_steps [n_sim_steps x 1] indices
%

```

```

% OPTIONAL INPUTS:
%   sim_results.h2_kgph          [n_sim_steps x 1]
%
% OUTPUTS:
%   emissions.co2_kg_per_timestep
%   emissions.total_co2_kg
%   emissions.co2_per_MWh_electric
%   emissions.co2_per_kg_h2      (if H2 supplied)
%   emissions.co2_per_kg_h2_timestep (if H2 supplied)
%
% UNITS:
%   Power          : kW
%   Energy         : MWh
%   Kernel alpha   : kg CO2 / MWh
% =====

dt_hr = inputs.dt_hr;

% -----
% Validate required fields
% -----
assert(isfield(sim_results, 'P_grid_kW'), ...
    'sim_results must contain P_grid_kW');

assert(isfield(emissions_kernel, 'alpha_kg_per_MWh'), ...
    'Emissions kernel missing: alpha_kg_per_MWh');

assert(nargin >= 4 && ~isempty(sim_steps), ...
    'sim_steps must be provided for time-windowed attribution');

% -----
% Slice kernel to simulated timesteps
% -----
alpha = emissions_kernel.alpha_kg_per_MWh(sim_steps);
P_grid_kW = sim_results.P_grid_kW(:);

assert(length(P_grid_kW) == length(alpha), ...
    'Mismatch: grid power and emissions kernel length after slicing');

% -----
% Convert power → energy
% -----
% kW × hr → kWh → MWh
energy_MWh = (P_grid_kW .* dt_hr) / 1000;

% -----
% CO2 emissions attribution
% -----
co2_kg_step = energy_MWh .* alpha;
total_co2_kg = sum(co2_kg_step);

% -----
% Package outputs
% -----

```

```

emissions.co2_kg_per_timestep = co2_kg_step;
emissions.total_co2_kg        = total_co2_kg;
emissions.co2_per_MWh_electric = ...
    safe_divide(total_co2_kg, sum(energy_MWh));

% -----
% Optional hydrogen intensity metrics
% -----
if isfield(sim_results, 'h2_kgph')
    h2_kg_step = sim_results.h2_kgph(:) .* dt_hr;

    emissions.co2_per_kg_h2 = ...
        safe_divide(total_co2_kg, sum(h2_kg_step));

    emissions.co2_per_kg_h2_timestep = ...
        safe_divide(co2_kg_step, h2_kg_step);
end

% -----
% Traceability metadata
% -----
emissions.simulated_timesteps = sim_steps;
if isfield(sim_results, 'time')
    emissions.simulated_time = sim_results.time;
end
emissions.kernel_used = emissions_kernel;

end

# FILE: src/matlab/models/define_electrolyzer_system.m
-----
function electrolyzer_system = define_electrolyzer_system()

electrolyzer_system.stack = define_stack_model();
electrolyzer_system.smps  = define_smps_model();
electrolyzer_system.psa   = define_psa_model();
electrolyzer_system.aux   = define_aux_model();

electrolyzer_system.physics = @(P_stack_kW) ...
    evaluate_system_physics( ...
        P_stack_kW, ...
        electrolyzer_system.stack, ...
        electrolyzer_system.smps, ...
        electrolyzer_system.psa, ...
        electrolyzer_system.aux );

end

% =====
% SYSTEM PHYSICS (VECTORIZED, FAST)
% =====

function physics = evaluate_system_physics(P_stack_kW, stack, smps, psa, aux)

```

```

% Ensure column vector
P = P_stack_kW(:);
N = numel(P);

% -----
% Physical guardrails (no policy)
% -----
P = max(P, 0);
P = min(P, stack.P_max_kW);

% -----
% OFF mask (AUTHORITATIVE)
% -----
is_on = P > 0;

% -----
% Preallocate outputs (struct-of-arrays)
% -----
physics.P_stack_kW      = P;
physics.mH2_gross_kgph  = zeros(N,1);
physics.mH2_net_kgph    = zeros(N,1);
physics.mH2_loss_kgph   = zeros(N,1);
physics.P_grid_smeps_kW = zeros(N,1);
physics.P_aux_kW        = zeros(N,1);
physics.P_grid_total_kW = zeros(N,1);
physics.eta_system_LHV  = zeros(N,1);

% -----
% Stack electrochemistry (ONLY WHEN ON)
% -----
physics.mH2_gross_kgph(is_on) = ...
    stack.mH2_gross_kgph(P(is_on));

% -----
% PSA purification (ONLY WHEN ON)
% -----
physics.mH2_net_kgph(is_on) = ...
    psa.mH2_net_kgph(physics.mH2_gross_kgph(is_on));

physics.mH2_loss_kgph(is_on) = ...
    psa.mH2_loss_actual_kgph(physics.mH2_gross_kgph(is_on));

% -----
% SMPS AC→DC electrical behavior (ONLY WHEN ON)
% -----
physics.P_grid_smeps_kW(is_on) = ...
    smeps.P_grid_kW(P(is_on));

P_smeps_loss = physics.P_grid_smeps_kW - P;

% -----
% Auxiliary loads (ONLY WHEN ON)
% -----

```

```

physics.P_aux_kW(is_on) = aux.P_total_kW();

% -----
% Total grid-side power (ONLY WHEN ON)
% -----
physics.P_grid_total_kW(is_on) = ...
    physics.P_grid_smeps_kW(is_on) ...
    + physics.P_aux_kW(is_on) ...
    + psa.P_parasitic_kW(physics.mH2_gross_kgph(is_on));

% -----
% Chemical power (LHV)
% -----
P_H2_net_LHV_kW = zeros(N,1);
P_H2_net_LHV_kW(is_on) = ...
    physics.mH2_net_kgph(is_on) .* stack.LHV_H2_MJ_per_kg / 3.6;

% -----
% System efficiency (authoritative)
% -----
valid = physics.P_grid_total_kW > 0;
physics.eta_system_LHV(valid) = ...
    P_H2_net_LHV_kW(valid) ./ physics.P_grid_total_kW(valid);

% -----
% Hierarchical grouping (diagnostics-friendly)
% -----
physics.stack.P_dc_kW          = physics.P_stack_kW;
physics.stack.mH2_gross_kgph = physics.mH2_gross_kgph;

physics.device.P_grid_kW      = physics.P_grid_smeps_kW;
physics.device.P_loss_kW      = P_smeps_loss;

physics.psa.mH2_net_kgph      = physics.mH2_net_kgph;
physics.psa.mH2_loss_kgph     = physics.mH2_loss_kgph;

physics.system.P_grid_kW      = physics.P_grid_total_kW;
physics.system.eta_LHV        = physics.eta_system_LHV;

% -----
% HARD PHYSICAL ASSERT
% -----
idx = physics.P_stack_kW == 0;

assert( ...
    all(physics.P_grid_total_kW(idx) == 0), ...
    'Physics violation: non-zero system power when stack is OFF' ...
);

end

% =====
% SYSTEM COMPONENTS

```

```

% =====
function stack = define_stack_model()

stack.name = "PEM_60kW_stack";

stack.P_rated_kW      = 60;
stack.P_max_kW        = 52;
stack.turndown_ratio  = 0.30;
stack.P_min_kW        = stack.P_max_kW * stack.turndown_ratio;

stack.P_axis_kW = stack.P_max_kW * [1.0 0.9 0.8 0.7 0.6 0.5 0.4 0.3]';
stack.mH2_axis   = [0.89 0.82 0.73 0.66 0.58 0.49 0.40 0.32]';

assert(all(diff(stack.P_axis_kW) < 0));
assert(all(diff(stack.mH2_axis) < 0));

stack.mH2_gross_kgph = @(P) ...
    interp1(stack.P_axis_kW, stack.mH2_axis, P, 'pchip', 0);

stack.HHV_H2_MJ_per_kg = 141.88;
stack.LHV_H2_MJ_per_kg = 119.96;

end

function smps = define_smeps_model()

smeps.name = "SMPS_ACDC_interface";

smeps.P_axis_kW = [5 10 20 30 40 50 60]';
smeps.eta_axis  = [0.80 0.86 0.91 0.92 0.925 0.93 0.935]';

assert(all(diff(smeps.P_axis_kW) > 0));

smeps.eta = @(P) ...
    interp1(smeps.P_axis_kW, smeps.eta_axis, P, 'pchip', smeps.eta_axis(1));

smeps.P_grid_kW = @(P_stack_kW) P_stack_kW ./ smeps.eta(P_stack_kW);

end

function psa = define_psa_model()

psa.name = "PSA_purification_system";

psa.mH2_loss_kgph_nominal = 0.12;

psa.mH2_net_kgph = @(m) max(m - psa.mH2_loss_kgph_nominal, 0);
psa.mH2_loss_actual_kgph = @(m) min(psa.mH2_loss_kgph_nominal, m);

psa.P_parasitic_kW = @(~) 0;

end

function aux = define_aux_model()

```

```

aux.name = "Auxiliary_systems";

aux.P_aux_kW      = 2.0;
aux.P_chiller_kW  = 17.0;

aux.P_total_kW    = @( ) aux.P_aux_kW + aux.P_chiller_kW;

end

# FILE: src/matlab/models/initialize_electrolyzer_system_state.m
-----
function state = initialize_electrolyzer_system_state()
%INITIALIZE_ELECTROLYZER_SYSTEM_STATE
% Optimized supervisory state initializer for electrolyzer system
%
% Design principles:
%   - Numeric enums instead of strings (performance)
%   - No dynamic field creation after initialization
%   - Scalar-only persistent state
%   - Parameter names aligned with CSV controller cases
%
% Enum encodings:
%
%   mode:
%       0 = OFF
%       1 = ON
%
%   operating_regime:
%       0 = NORMAL
%       1 = TURNDOWN
%
%   constraint_type:
%       0 = none
%       1 = MIN_UP      (min_up_steps)
%       2 = MIN_DOWN    (min_down_steps)

% -----
% Availability & operating regime (ENUMS)
% -----
state.mode          = uint8(0);    % OFF
state.operating_regime = uint8(0); % NORMAL
state.is_operating   = false;      % cached boolean for convenience

% -----
% Runtime tracking (timesteps)
% -----
state.runtime_steps = uint32(0);
state.offtime_steps = uint32(0);

state.startups      = uint32(0);
state.shutdowns     = uint32(0);

```

```

% -----
% Regime persistence counters
% (associated with turndown_delay and recover_delay)
% -----
state.turndown_counter = uint32(0);
state.recover_counter  = uint32(0);

% -----
% Price-stop persistence counter
% (associated with price_delay)
% -----
state.price_violation_counter = uint32(0);

% -----
% ON constraint (minimum runtime enforcement)
% Corresponds to: min_up_steps
% -----
state.on_constraint_active      = false;
state.on_constraint_type        = uint8(0); % none | MIN_UP
state.on_constraint_steps_remaining = uint32(0);

% -----
% OFF constraint (minimum downtime enforcement)
% Corresponds to: min_down_steps
% -----
state.off_constraint_active      = false;
state.off_constraint_type        = uint8(0); % none | MIN_DOWN
state.off_constraint_steps_remaining = uint32(0);

% -----
% Cumulative accounting (non-decision, diagnostic only)
% -----
state.h2_produced_kg = 0.0;
state.energy_grid_kWh = 0.0;

end

# FILE: src/matlab/models/update_electrolyzer_system_state.m
-----
function state = update_electrolyzer_system_state( ...
    state, current_case, stack, signals, jj, use_discrete_dispatch)

% ===== ENUMS =====
MODE_OFF = uint8(0);
MODE_ON  = uint8(1);

REG_NORMAL  = uint8(0);
REG_TURNDOWN = uint8(1);

% ===== SIGNALS =====
price      = signals.price(jj);
clean_ratio = signals.clean_ratio(jj);

```

```

% ===== PRICE DELAY =====
if price > current_case.price_stop
    state.price_violation_counter = state.price_violation_counter + 1;
else
    state.price_violation_counter = uint32(0);
end

price_allowed = ...
    state.price_violation_counter < current_case.price_delay;

% ===== CONDITIONS =====
can_start = ...
    clean_ratio >= current_case.clean_ratio_start && ...
    price <= current_case.price_start;

stop_conditions = ...
    clean_ratio < current_case.clean_ratio_stop || ...
    ~price_allowed;

bad_conditions = ...
    clean_ratio < current_case.clean_ratio_turndown || ...
    price > current_case.price_turndown;

good_conditions = ...
    clean_ratio >= current_case.clean_ratio_start && ...
    price <= current_case.price_start;

% ===== STATE TRANSITIONS =====
if state.mode == MODE_ON

    state.runtime_steps = state.runtime_steps + 1;
    state.offtime_steps = uint32(0);

    if bad_conditions
        state.turndown_counter = state.turndown_counter + 1;
        state.recover_counter = uint32(0);
    else
        state.turndown_counter = uint32(0);
    end

    if good_conditions
        state.recover_counter = state.recover_counter + 1;
    else
        state.recover_counter = uint32(0);
    end

    if state.operating_regime == REG_NORMAL && ...
        state.turndown_counter >= current_case.turndown_delay
        state.operating_regime = REG_TURNDOWN;
    end

    if state.operating_regime == REG_TURNDOWN && ...
        state.recover_counter >= current_case.recover_delay
        state.operating_regime = REG_NORMAL;
    end
end

```

```

end

if state.operating_regime == REG_TURNDOWN && ...
    state.runtime_steps >= current_case.min_up_steps && ...
    stop_conditions

    state.mode                = MODE_OFF;
    state.operating_regime    = REG_NORMAL;
    state.is_operating        = false;
    state.shutdowns           = state.shutdowns + 1;

    state.runtime_steps       = uint32(0);
    state.turndown_counter    = uint32(0);
    state.recover_counter     = uint32(0);
end

else
    % ----- OFF STATE -----
    state.offtime_steps = state.offtime_steps + 1;
    state.runtime_steps = uint32(0);

    state.operating_regime = REG_NORMAL;
    state.turndown_counter = uint32(0);
    state.recover_counter  = uint32(0);
    state.price_violation_counter = uint32(0);

    if state.offtime_steps >= current_case.min_down_steps && can_start
        state.mode                = MODE_ON;
        state.is_operating        = true;
        state.startups            = state.startups + 1;
    end
end

% ===== POWER COMMAND (FINAL & AUTHORITATIVE) =====
state.P_stack_cmd_kw = 0.0; % ← DEFAULT TO ZERO

if state.mode == MODE_ON
    if state.operating_regime == REG_NORMAL
        state.P_stack_cmd_kw = stack.P_max_kw;

    elseif state.operating_regime == REG_TURNDOWN
        if use_discrete_dispatch
            state.P_stack_cmd_kw = stack.P_min_kw;
        else
            frac = (clean_ratio - current_case.clean_ratio_stop) / ...
                (current_case.clean_ratio_start - current_case.clean_ratio_stop);
            frac = min(max(frac, 0), 1);
            state.P_stack_cmd_kw = ...
                stack.P_min_kw + frac * ...
                (stack.P_max_kw - stack.P_min_kw);
        end
    end
end
end

```

```

% ===== HARD ASSERT =====
assert(~(state.mode == MODE_OFF && state.P_stack_cmd_kW > 0), ...
    'CRITICAL: OFF mode with non-zero power at timestep %d', jj);

end

# FILE: src/matlab/io/init_results_hdf5_chunked.m
-----
function init_results_hdf5_chunked(outFile, nCases, time, chunkSize)

if exist(outFile, 'file')
    delete(outFile);
end

time = time(:);
nTime = numel(time);
chunkCases = min(chunkSize, nCases);

% --- Time (static, contiguous)
h5create(outFile, '/time', [nTime 1], 'Datatype', 'double');
h5write(outFile, '/time', posixtime(time));

% --- Metadata
create('/case_id', [nCases 1], [chunkCases 1]);

% --- Time series
create('/sim/P_grid_kW', [nCases nTime], [chunkCases nTime]);
create('/sim/h2_kgph', [nCases nTime], [chunkCases nTime]);
create('/sim/cost', [nCases nTime], [chunkCases nTime]);

% --- Emissions
create('/emissions/co2_kg_per_timestep', [nCases nTime], [chunkCases nTime]);

% --- Totals
create('/totals/total_energy_MWh', [nCases 1], [chunkCases 1]);
create('/totals/total_cost', [nCases 1], [chunkCases 1]);
create('/totals/total_h2_kg', [nCases 1], [chunkCases 1]);
create('/totals/total_co2_kg', [nCases 1], [chunkCases 1]);

% --- State
create('/state/startups', [nCases 1], [chunkCases 1]);

function create(name, sz, chunk)
    h5create(outFile, name, sz, ...
        'ChunkSize', chunk, 'Datatype', 'double');
end

end

# FILE: src/matlab/io/write_results_hdf5_chunk.m
-----
function write_results_hdf5_chunk(outFile, buffer, iStart)

n = numel(buffer);
nTime = numel(buffer(1).sim.P_grid_kW);

```

```

case_id = zeros(n,1);
P_grid_kW = zeros(n,nTime);
h2_kgph = zeros(n,nTime);
cost = zeros(n,nTime);
co2_kg = zeros(n,nTime);

E = zeros(n,1);
C = zeros(n,1);
H = zeros(n,1);
CO2 = zeros(n,1);
S = zeros(n,1);

for k = 1:n
    r = buffer(k);
    case_id(k) = r.ID;
    P_grid_kW(k,:) = r.sim.P_grid_kW.';
    h2_kgph(k,:) = r.sim.h2_kgph.';
    cost(k,:) = r.sim.cost.';
    co2_kg(k,:) = r.emissions.co2_kg_per_timestep.';
    E(k) = r.sim.total_energy_MWh;
    C(k) = r.sim.total_cost;
    H(k) = r.sim.total_h2_kg;
    CO2(k) = r.emissions.total_co2_kg;
    S(k) = r.state.startups;
end

iEnd = iStart + n - 1;

h5write(outFile, '/case_id', case_id, [iStart 1], [n 1]);
h5write(outFile, '/sim/P_grid_kW', P_grid_kW, [iStart 1], [n nTime]);
h5write(outFile, '/sim/h2_kgph', h2_kgph, [iStart 1], [n nTime]);
h5write(outFile, '/sim/cost', cost, [iStart 1], [n nTime]);
h5write(outFile, '/emissions/co2_kg_per_timestep', co2_kg, [iStart 1], [n nTime]);

h5write(outFile, '/totals/total_energy_MWh', E, [iStart 1], [n 1]);
h5write(outFile, '/totals/total_cost', C, [iStart 1], [n 1]);
h5write(outFile, '/totals/total_h2_kg', H, [iStart 1], [n 1]);
h5write(outFile, '/totals/total_co2_kg', CO2, [iStart 1], [n 1]);
h5write(outFile, '/state/startups', S, [iStart 1], [n 1]);
end

# FILE: src/matlab/io/export_results_hdf5_chunked.m
-----
function export_results_hdf5_chunked(results, outFile, chunkSize)
% EXPORT_RESULTS_HDF5_CHUNKED
%
% Writes results to HDF5 using chunked datasets.
% Case-wise chunking enables scalable I/O and future streaming.
%
% Layout is identical to export_results_hdf5.
%

    if nargin < 3

```

```

        chunkSize = 20;
end

if exist(outFile, 'file')
    delete(outFile);
end

nCases = numel(results);
time    = results(1).sim.time(:);
nTime   = numel(time);

% Ensure chunk size is valid
chunkCases = min(chunkSize, nCases);

%% -----
% Initialize HDF5 file and datasets
% -----

% --- Static time vector (contiguous, no chunking)
h5create(outFile, '/time', [nTime 1], 'Datatype', 'double');
h5write(outFile, '/time', datetime_to_posixtime(time));

% --- Case metadata
create_dataset(outFile, '/case_id',      [nCases 1],      [chunkCases 1],      'double');

% --- Simulation time series
create_dataset(outFile, '/sim/P_grid_kW', [nCases nTime], [chunkCases nTime], 'double');
create_dataset(outFile, '/sim/h2_kgph',   [nCases nTime], [chunkCases nTime], 'double');
create_dataset(outFile, '/sim/cost',      [nCases nTime], [chunkCases nTime], 'double');

% --- Emissions
create_dataset(outFile, '/emissions/co2_kg_per_timestep', ...
                [nCases nTime], [chunkCases nTime], 'double');

% --- Totals
create_dataset(outFile, '/totals/total_energy_MWh', [nCases 1], [chunkCases 1], 'double');
create_dataset(outFile, '/totals/total_cost',       [nCases 1], [chunkCases 1], 'double');
create_dataset(outFile, '/totals/total_h2_kg',      [nCases 1], [chunkCases 1], 'double');
create_dataset(outFile, '/totals/total_co2_kg',     [nCases 1], [chunkCases 1], 'double');

% --- State
create_dataset(outFile, '/state/startups', [nCases 1], [chunkCases 1], 'double');

%% -----
% Write data in case chunks
% -----
for i1 = 1:chunkCases:nCases
    i2 = min(i1 + chunkCases - 1, nCases);
    n   = i2 - i1 + 1;

    % Buffers
    case_id = zeros(n,1);

    P_grid_kW = zeros(n,nTime);

```

```

h2_kgph    = zeros(n,nTime);
cost       = zeros(n,nTime);
co2_kg     = zeros(n,nTime);

total_energy_MWh = zeros(n,1);
total_cost      = zeros(n,1);
total_h2_kg     = zeros(n,1);
total_co2_kg    = zeros(n,1);
startups       = zeros(n,1);

% Fill buffers
for k = 1:n
    r = results(i1 + k - 1);

    case_id(k) = r.ID;

    P_grid_kW(k,:) = r.sim.P_grid_kW(:).';
    h2_kgph(k,:)   = r.sim.h2_kgph(:).';
    cost(k,:)      = r.sim.cost(:).';
    co2_kg(k,:)    = r.emissions.co2_kg_per_timestep(:).';

    total_energy_MWh(k) = r.sim.total_energy_MWh;
    total_cost(k)       = r.sim.total_cost;
    total_h2_kg(k)      = r.sim.total_h2_kg;
    total_co2_kg(k)     = r.emissions.total_co2_kg;
    startups(k)        = r.state.startups;
end

% Write chunk
h5write(outFile, '/case_id', case_id, [i1 1], [n 1]);

h5write(outFile, '/sim/P_grid_kW', P_grid_kW, [i1 1], [n nTime]);
h5write(outFile, '/sim/h2_kgph',   h2_kgph,   [i1 1], [n nTime]);
h5write(outFile, '/sim/cost',      cost,      [i1 1], [n nTime]);

h5write(outFile, '/emissions/co2_kg_per_timestep', ...
        co2_kg, [i1 1], [n nTime]);

h5write(outFile, '/totals/total_energy_MWh', total_energy_MWh, [i1 1], [n 1]);
h5write(outFile, '/totals/total_cost',      total_cost,      [i1 1], [n 1]);
h5write(outFile, '/totals/total_h2_kg',     total_h2_kg,     [i1 1], [n 1]);
h5write(outFile, '/totals/total_co2_kg',    total_co2_kg,    [i1 1], [n 1]);

h5write(outFile, '/state/startups', startups, [i1 1], [n 1]);
end

fprintf('Chunked HDF5 results written to:\n  %s\n', outFile);
end

%% =====
% Helper functions
% =====

function create_dataset(file, name, sz, chunkSz, datatype)

```

```

        h5create(file, name, sz, ...
            'Datatype', datatype, ...
            'ChunkSize', chunkSz);
end

function t = datetime_to_posixtime(dt)
    t = posixtime(dt);
end

# FILE: src/matlab/io/export_results_timeseries.m
-----
function export_results_timeseries(results, varList, outFile)
% EXPORT_RESULTS_TIMESERIES
%
% Export selected time-series variables from results struct to a flat table
% suitable for Python (pandas) plotting.
%
% Inputs:
%   results - struct array (results(ii))
%   varList - string array or cellstr of dot-paths
%             e.g. ["sim.P_grid_kW", "sim.h2_kgph", "emissions.co2_kg_per_timestep"]
%   outFile - output CSV filename
%
% Output:
%   Writes CSV to disk

    if ischar(varList)
        varList = {varList};
    end

    rows = []; %%ok<NASGU>
    allTables = cell(numel(results),1);

    for ii = 1:numel(results)

        r = results(ii);
        t = r.sim.time;
        n = numel(t);

        T = table();
        T.case_id = repmat(r.ID, n, 1);
        T.time = t;

        for v = 1:numel(varList)
            path = varList{v};

            data = get_nested_field(r, path);

            if numel(data) ~= n
                error('Variable "%s" is not a time-series of length %d.', ...
                    path, n);
            end

            varName = matlab.lang.makeValidName(strrep(path, '.', '_'));

```

```

        T.(varName) = data;
    end

    allTables{ii} = T;
end

outTable = vertcat(allTables{:});
writetable(outTable, outFile);

fprintf('Exported %d rows to %s\n', height(outTable), outFile);
end
function value = get_nested_field(s, path)
% GET_NESTED_FIELD Access struct fields using dot notation
%
% Example:
%   get_nested_field(results(1), 'sim.P_grid_kw')

    parts = strsplit(path, '.');
    value = s;

    for i = 1:numel(parts)
        field = parts{i};
        if ~isfield(value, field)
            error('Field "%s" not found in path "%s".', field, path);
        end
        value = value.(field);
    end
end

# FILE: src/matlab/utils/get_project_root.m
-----
function root = get_project_root()
% GET_PROJECT_ROOT Returns absolute path to project root
%
% Project root is identified by presence of .project-root file.
% This makes paths robust to pwd changes and machine differences.

    persistent cached_root

    if ~isempty(cached_root)
        root = cached_root;
        return
    end

    % Start from this file's location (not pwd)
    here = fileparts(mfilename('fullpath'));

    % Walk upward until we find .project-root
    root = here;
    while true
        marker = fullfile(root, '.project-root');
        if isfile(marker)
            cached_root = root;
            return
        end
    end
end

```

```

        end

        parent = fileparts(root);
        if isempty(parent) || strcmp(parent, root)
            error('Project root not found. Missing .project-root file.');
```

```

        end

        root = parent;
    end
end

# FILE: src/matlab/utils/safe_divide.m
-----
function y = safe_divide(a,b)
y = zeros(size(a));
mask = b > 0;
y(mask) = a(mask) ./ b(mask);
end

# FILE: src/matlab/tools/printStructStructure.m
-----
function printStructStructure(s, indent)
% PRINTSTRUCTSTRUCTURE Pretty-prints the STRUCTURE of a struct (not values)
%
%   printStructStructure(s)
%   printStructStructure(s, indent)
%
% Shows:
%   - field names
%   - data types
%   - sizes
%   - nesting
%   - struct and cell hierarchy
%
% Intended for config inspection and debugging.

    if nargin < 2
        indent = 0;
    end

    if ~isstruct(s)
        error('Input must be a struct.');
```

```

    end

    spacer = repmat(' ', 1, indent);

    % Handle struct arrays
    if numel(s) > 1
        sz = size(s);
        fprintf('%sstruct array %s\n', spacer, mat2str(sz));
        fprintf('%sFields:\n', spacer);
        s = s(1);          % assume identical structure
        indent = indent + 4;
        spacer = repmat(' ', 1, indent);
    end
end

```

```

end

fields = fieldnames(s);

if isempty(fields)
    fprintf('%s(empty struct)\n', spacer);
    return
end

for i = 1:numel(fields)
    field = fields{i};
    value = s.(field);

    typeStr = class(value);
    sizeStr = mat2str(size(value));

    fprintf('%s■■ %s (%s, %s)\n', spacer, field, typeStr, sizeStr);

    if isstruct(value)
        printStructStructure(value, indent + 4);

    elseif iscell(value)
        printCellStructure(value, indent + 4);
    end
end
end

function printCellStructure(c, indent)
% PRINTCELLSTRUCTURE Prints structural info for cell arrays (not contents)

    spacer = repmat(' ', 1, indent);
    sz = size(c);

    fprintf('%scell array %s\n', spacer, mat2str(sz));

    if isempty(c)
        fprintf('%s(empty cell)\n', spacer);
        return
    end

    % Inspect first element only (representative)
    first = c{1};

    fprintf('%s■■ {1} → %s (%s)\n', ...
        spacer, class(first), mat2str(size(first)));

    if isstruct(first)
        printStructStructure(first, indent + 4);
    elseif iscell(first)
        printCellStructure(first, indent + 4);
    end
end
end

```

```

# FILE: src/matlab/shutdown.m
-----
function shutdown()
%SHUTDOWN Cleanup for ERCOT Electrolyzer Sim Engine
%
% Restores MATLAB environment to a clean state.
% Intended to leave no side effects after project use.

    fprintf('[shutdown] Shutting down ERCOT Electrolyzer Sim Engine...\n');

    %-----
    % Clear persistent variables & classes
    %-----
    clear functions
    clear classes

    %-----
    % Restore MATLAB default path
    %-----
    restoredefaultpath;

    %-----
    % Close figures created during engine use (if any)
    %-----
    close all force

    fprintf('[shutdown] Environment restored to MATLAB defaults.\n');

end

```